Project APEX

Chapter 23 — Backend Microservices

23.1 Purpose

The Backend Microservices architecture decomposes Project APEX into independently deployable services. Each service owns a specific business capability, exposes well-defined APIs, and communicates through synchronous APIs or asynchronous events.

23.2 Objectives

• Modular architecture
• Independent deployment
• Service isolation
• Horizontal scalability
• Fault tolerance

23.3 Core Services

• API Gateway Service
• Identity Service
• Observation Service
• Memory Service
• Knowledge Graph Service
• Retrieval Service
• Planning Service
• Execution Service
• Plugin Service
• Monitoring Service

23.4 Communication

Services communicate using REST/gRPC for synchronous requests and an event bus for asynchronous workflow events. Each service maintains its own data ownership and publishes domain events.

23.5 Service Principles

• Database per service
• Loose coupling
• High cohesion
• Contract-first APIs
• Event-driven integration

23.6 Deployment Model

Every service is packaged as an independent container, versioned separately, and deployed through automated CI/CD pipelines. Services can be scaled individually based on workload.

23.7 Challenges

Distributed systems introduce network latency, partial failures, data consistency challenges, service discovery requirements, and operational complexity.

23.8 Role in APEX

The microservice layer provides the production backbone for Project APEX by separating AI, storage, observation, execution, and infrastructure concerns into maintainable engineering components.

Service	Primary Responsibility
API Gateway	Entry point & routing
Identity	Authentication & authorization
Observation	Capture workflow events
Memory	Long-term knowledge
Knowledge Graph	Semantic relationships
Planning	Execution planning
Execution	Tool invocation
Monitoring	Logs, metrics & health

23.9 Chapter Summary

The Backend Microservices architecture provides a scalable, maintainable, and resilient engineering foundation for Project APEX. Clear service boundaries and event-driven communication enable independent evolution of platform capabilities.